

# Lab 1: Introduction to Android App Development

---

Android Studio • Project Creation • Basic UI • Android Emulator

*Lab Tutorial*

## Lab Overview

In this lab, you will become familiar with Android Studio and create a simple Android application. The application will display a text label and a button. When the button is pressed, the text displayed by the label will change. You will then run and test the application using an Android Virtual Device (AVD).

## Learning Outcomes

- Identify the main areas of the Android Studio development environment.
- Create and configure a new Android application project.
- Recognize the basic structure of an Android project.
- Create a simple user interface containing a TextView (label) and Button.
- Add basic button-click behaviour using Kotlin.
- Create or select an Android Virtual Device and run the application in the Android Emulator.
- Perform a basic functional test of the application.

## Prerequisites and Software

This tutorial assumes that Android Studio is installed with the Android SDK and Emulator components. The exact Android Studio screens may differ slightly by version; use the closest equivalent option if a label has changed.

| Requirement            | Purpose                                                                                              |
|------------------------|------------------------------------------------------------------------------------------------------|
| Android Studio         | Integrated development environment (IDE) used to create, build, run, and debug Android applications. |
| Android SDK            | Development tools and Android platform files required to build applications.                         |
| Android Emulator / AVD | A simulated Android device used to run and test the application.                                     |
| Kotlin                 | Programming language used for the application logic in this tutorial.                                |

## Part 1 - Become Familiar with Android Studio

Android Studio is the official IDE for Android development. A typical project window contains several important areas:

- **Project tool window:** Displays project files and folders. In the Android view, files are grouped into convenient Android-specific categories.
- **Editor:** The main area in which Kotlin, XML, and other project files are edited.

- **Toolbar:** Provides common actions such as selecting a device, running the app, debugging, and building the project.
- **Tool windows:** Panels such as Logcat, Build, Device Manager, and Terminal provide diagnostic and development tools.
- **Device Manager:** Used to create and manage Android Virtual Devices and connected physical devices.

Tip: Android Studio may download SDK components or Gradle dependencies the first time a project is created. Allow these processes to finish before troubleshooting build errors.

## Part 2 - Create Your First Android Project

1. Open Android Studio. From the Welcome screen, select New Project. If another project is already open, use File > New > New Project.
2. Select a simple phone/tablet template that creates a Views-based application, such as Empty Views Activity. Avoid a Jetpack Compose template for this introductory exercise.
3. Configure the project using the values below. The exact minimum SDK offered may vary with your installed SDK.

| Setting       | Suggested value                                                              |
|---------------|------------------------------------------------------------------------------|
| Name          | Lab1HelloAndroid                                                             |
| Package name  | Use the generated package name or your course naming convention              |
| Save location | A folder in your course/lab workspace                                        |
| Language      | Kotlin                                                                       |
| Minimum SDK   | Use the course/instructor requirement; otherwise keep the reasonable default |

4. Select Finish/Create and wait for Android Studio to create and synchronize the project.
5. Confirm that the project opens without a build error before continuing.

## Part 3 - Examine the Project Structure

In the Project tool window, locate the app module. The following files are especially important in this lab:

- **MainActivity.kt:** Contains Kotlin code for the main screen and will handle the button click.
- **res/layout/activity\_main.xml:** Defines the screen layout when using the traditional Android Views system.
- **res/values/strings.xml:** Stores user-visible text resources.
- **AndroidManifest.xml:** Describes essential application components and configuration.
- **Gradle build files:** Define build configuration and project dependencies.

## Part 4 - Build a Basic User Interface

Open app > res > layout > activity\_main.xml. Android Studio can show a Design view, Code view, or Split view. For this exercise, use the visual Layout Editor or XML. The goal is a screen with one TextView and one Button.

### Option A - Use the Layout Editor

6. Open activity\_main.xml and select Design or Split.
7. If necessary, remove the default sample TextView.

- From the Palette, drag a TextView onto the layout. Set its id to textViewMessage and its displayed text to Hello Android!
- Drag a Button below the TextView. Set its id to buttonChangeText and its text to Click Me.
- If the root layout is ConstraintLayout, add constraints so both controls have horizontal and vertical positions. Do not leave constraint warnings unresolved.

## Option B - Enter the Layout as XML

You may instead replace the contents of activity\_main.xml with the following simple ConstraintLayout:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textViewMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello Android!"
        android:textSize="24sp"
        app:layout_constraintBottom_toTopOf="@+id/buttonChangeText"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <Button
        android:id="@+id/buttonChangeText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Click Me"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/textViewMessage" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Note: Android Studio may recommend moving hard-coded text such as “Hello Android!” and “Click Me” into strings.xml. That is a good Android practice and can be completed as an extension at the end of the lab.

## Part 5 - Add Button Behaviour with Kotlin

Open MainActivity.kt. The activity's onCreate() method runs when the screen is created. After setContentView(...) loads the layout, obtain references to the TextView and Button and register a click listener.

```
package your.package.name

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```

setContentView(R.layout.activity_main)

val message = findViewById<TextView>(R.id.textViewMessage)
val button = findViewById<Button>(R.id.buttonChangeText)

button.setOnClickListener {
    message.text = "Button clicked!"
}
}
}

```

Important: keep the package declaration generated for your own project. Replace only the activity code as needed; do not copy the placeholder package name shown above.

## Part 6 - Create or Select an Android Virtual Device

An Android Virtual Device (AVD) defines the hardware profile and Android system image used by the Android Emulator.

11. Open Device Manager from Android Studio (for example, Tools > Device Manager).
12. If an AVD already exists, you may use it. Otherwise, choose the option to create a virtual device.
13. Select a phone hardware profile, such as a recent Pixel profile, and continue.
14. Select an Android system image. If the image is not installed, download it when prompted. Use an image compatible with your computer.
15. Accept or review the AVD configuration, give the device a recognizable name if desired, and finish creating it.
16. Start the virtual device from Device Manager and wait until the Android home screen is fully loaded.

## Part 7 - Run the Application in the Emulator

17. In the Android Studio toolbar, confirm that the app run configuration is selected.
18. Select your running AVD from the target device selector.
19. Click Run (the triangle/play icon). Android Studio will build the project, install the app on the emulator, and launch it.
20. Confirm that the screen displays the Hello Android! label and the Click Me button.
21. Press Click Me. Confirm that the label changes to Button clicked!.

## Part 8 - Basic Troubleshooting

| Problem                  | What to check                                                                                                                        |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Project does not build   | Check the Build output for the first error. Wait for Gradle synchronization and required SDK downloads to complete.                  |
| No emulator is available | Open Device Manager and create an AVD with an installed system image.                                                                |
| Emulator starts slowly   | Allow the first boot to complete. Later starts are usually faster. Ensure hardware virtualization is available on supported systems. |
| App does not appear      | Confirm the correct target device is selected and review the Run/Build output for installation errors.                               |
| Button does nothing      | Verify that the IDs in MainActivity.kt exactly match                                                                                 |

|                               |                                                                              |
|-------------------------------|------------------------------------------------------------------------------|
|                               | the IDs in activity_main.xml and that setContentView() loads activity_main.  |
| Layout controls are misplaced | If using ConstraintLayout, check that each view has appropriate constraints. |

## Part 9 - Lab Checkpoint

Before completing the lab, demonstrate or verify all of the following:

- ☐ The Android Studio project opens and builds successfully.
- ☐ The application contains one visible TextView and one visible Button.
- ☐ The application runs in an Android Emulator.
- ☐ The initial TextView displays Hello Android!.
- ☐ Pressing the button changes the TextView to Button clicked!.
- ☐ You can identify MainActivity.kt, activity\_main.xml, strings.xml, and AndroidManifest.xml in the project.

## Submission / Evidence

Create a GIT repository for the this project and add code to the repository.

Make sure you add Przemyslaw Pawluk (ppawluk) as collaborator.

Submit link to the repository.

Submit a video (screen capture) of the application working in the emulator.

## Reference Resources

Official Android Developers resources:

- **Install Android Studio:** <https://developer.android.com/studio/install>
- **Create a project:** <https://developer.android.com/studio/projects/create-project>
- **Build a simple user interface:** <https://developer.android.com/develop/ui/views/layout/declaring-layout>
- **Create and manage virtual devices:** <https://developer.android.com/studio/run/managing-avds>
- **Run apps on the Android Emulator:** <https://developer.android.com/studio/run/emulator>

**End of Lab 1**